

코드화 패턴

비개발자가 AI를 다루는 새로운 패러다임

GPTers 22기 베스트 사례 발표

DY Cho (조셉)

"코드를 모르지만, 코드를 만들었습니다"

오프닝: Parker 의자 장면

"의자 = 방패와 창"

Parker(2013) 영화에서 의자는 단순한 가구가 아니었다.
방어의 방패이자 공격의 창이었다.

저는 AI를 그냥 '시켜 먹는' 도구로 썼습니다.
그러다 \$100이 하루만에 사라졌습니다.

AI도 마찬가지다.
도구로 소비하지 말고, 시스템으로 설계하라.

문제 정의: MD 강제성이 모델별로 달라서 토큰 비 낭비 커짐

Claude Opus 4		100%	\$15/\$75	강제성: 높음
GPT-5	8%		\$1.25/\$10	강제성: 높음
Gemini 2.5 Pro	8%		\$1.25/\$10	강제성: 높음
DeepSeek V4 Pro	3%		\$0.435/\$0.87	강제성: 중간
MiMo 2.5 Pro	3%		\$0.435/\$0.87	강제성: 낮음
코드화 (Python)	0% ★		\$0	강제성: -

* 비용 = Claude Opus 4 대비 상대적 토큰 비용 (%)

★ = 토큰비 0, 강제성 무관

AI 과금 구조 — 숨겨진 비용: Thinking 토큰

Input (입력): 사용자 프롬프트 → input 가격 **Output (출력):** 모델 응답 → output 가격

Thinking (추론): 모델의 내부 추론 → output 가격으로 과금 (input 아님!)

Thinking 포함 (\$0.27/회)

입력 1,000 + thinking 10,000 + 출력 500 토큰

\$100으로 약 374번 대화

이런 작업에 **thinking** 발생:

분석 · 기획 · 추론 · 창작
(사업 모델 분석, 코드 디버깅,
전략 수립, 이야기 창작 등)

Thinking 없이 (\$0.02/회)

입력 1,000 + 출력 500 토큰

\$100으로 약 5,714번 대화

이런 작업은 **thinking** 없음:

유틸리티 · 분류 · 설명 · 생성
(번역, 요약, 형식 변환,
용어 설명, 이름 추천 등)

15배 차이!

프레임워크도 코드화 가능 ZEN 큐브 6면 완성 여부, BOS 박스 분석 완료 여부 등 → 코드가 즉시 검증 → AI에게 입력값 전달 → AI는 내용 생성 · 전략 수립에 집중

왜 코드화인가? — 3가지 이익

Claude Opus 4 기준 상대적 토큰 비용

모델	상대 비용	MD 강제성	비고
Claude Opus 4	100%	★★★★★	기준. 비쌈
Claude Sonnet 4	20%	★★★★★	Opus와 동등 강제성
GPT-5 / Gemini Pro	8%	★★★★☆	높은 강제성
DeepSeek V4 Pro	3%	★★★☆☆	중간 강제성
MiMo 2.5 Pro	3%	★★☆☆☆	낮은 강제성
코드화 (Python)	0%	항상 동일	강제성 무관

1. 토큰비 절감

Claude Opus 대비 97% 절감
Thinking 토큰 비용 0

2. 검증 일관성

모델마다 다른 해석 → 코드화하면 통일
100% 동일한 결과 보장

3. 모델 변경 무관

새 모델이 나와도 코드화된 규칙 유지
Claude → MiMo 전환 시에도 문제 없음

코드화 패턴의 본질

코드화 =

AI가 반복 검증할 "입력값"을 만드는 것

= 검증 틀체인을 구축하는 것

= 비개발자의 시간·에너지를 절약하면서 AI의 판단력은 유지하는 것

최종값 패턴 (X)



최종 판정 (AI 판단 없음, 경직됨)

상황-맥락 무시
예외 판단 불가

입력값 패턴 (O)



상황-맥락 고려, 예외 판단 가능, 유연함

비개발자가 "절대적"이라 정의 안 해도 됨
규칙이 100% 정확하지 않아도 AI가 보완

공조검규: 코드화의 4가지 전제 조건

코드화할 수 있는 것 vs 없는 것

공식화

이 계산으로 결과를 냄

키워드 밀도 = 키워드 수 / 전체 단어 수 × 100

조건화

이 조건이면 통과, 아니면 위반

키워드 3개 이상이면 통과

검증화

맞다/틀리다로 판별할 수 있어야 함

코드화의 전제 조건 — 검증할 수 없는 것(감정, 창의성)은 코드화 불가

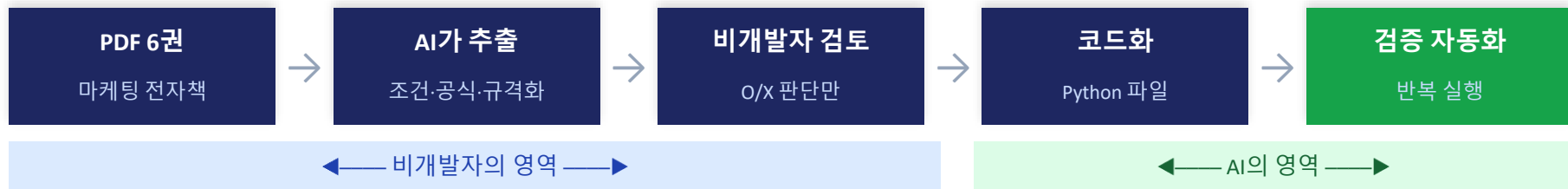
규격화

이 형식이어야 함

글자 수 500자 이상 2000자 이하

코드화 불가: 감정 · 창의성 · 맥락 판단 · 윤리적 해석 → AI의 판단력이 필요한 영역

실전 사례: marketing_rules.py



100X

토큰비 절감

6권

PDF 규칙 코드화

301줄

Python 코드

비개발자가 "이 규칙을 코드화해라"라고 말하면
AI가 코드를 만들고, 비개발자는 검토·결정만.
이 과정이 "비개발자의 코드화"의 원형이다.

실전 사례: 파일 의존성 구조

Before

메모리 (87%)

규칙 전체 저장 (13,151자)

모델마다 해석 다름
중복 관리



After

메모리 (55%)

"참조 한 줄"



Python 파일 (코드화)

모델 독립성 확보
중복 제거 완료

핵심: "이 규칙 → 이 파일" 한 줄로 축약 = 코드화 + 축약

코드화(2,443자) + 축약(2,326자) = 총 4,769자 절약

결론

"AI를 도구로 소비하지 말고 시스템으로 설계하라"

쉽게 말하면,
'시켜 먹지 말고 같이 일하라'는 겁니다.

비개발자가 "시스템을 설계"하는 경험
AI가 "검증 툴체인"을 구축하는 것
코드화된 결과가 "입력값"으로서 AI가 판단하는 것

감사합니다